

The Fastest Engine Is Not the Shippable Engine: Replacing a Regex Engine for Data Redaction Under Production Constraints

DANIELE FRISANCO, TODO: affiliation, TODO

We report on replacing the regex engine behind a data-redaction library to improve performance under four production constraints—zero runtime dependencies, byte-for-byte-identical output to the incumbent, robustness across all match densities, and thread-safety in a managed runtime—and find, across a sequence of prototypes, that the speed-optimal engine is ruled out by those constraints, and that the pre-filter pipeline usually assumed best for multi-pattern matching does not hold across densities; the best shippable design is a per-pattern lazy DFA with two domain-specific merges.

Additional Key Words and Phrases: regular expressions, multi-pattern matching, data redaction, lazy DFA, performance engineering, Ruby C extensions

1 INTRODUCTION

TODO. Hook + objective/constraints + contributions.

2 BACKGROUND

TODO.

3 THE CONSTRAINT BOX

The objective is performance, but it is performance constrained, and the constraints are what make the problem non-trivial—each one disqualifies an option that is otherwise attractive on speed alone. We treat three of them as binary gates: a candidate either passes or is ineligible, regardless of how fast it is. The fourth is the shape of the objective itself.

Gate 1: zero runtime dependencies. The library installs as a gem with no native dependencies beyond what Ruby itself links. This rules out the speed-optimal engine outright. Plain PCRE2 JIT is the fastest engine we measured at every density (§6), and it is disqualified not for being slow but for requiring a third-party shared library at run time. The gate converts the fastest option into a non-option, which is the paper’s thesis in miniature.

Gate 2: byte-for-byte-identical output. The replacement must redact exactly the same byte ranges as the incumbent, character for character, so that swapping the engine is invisible to every downstream consumer. This is the gate that kills the most tempting algorithmic shortcut. Merging all patterns into a single combined automaton—the textbook way to make multi-pattern matching one pass—was among the fastest prototypes we built (§4), but it does not preserve per-pattern semantics: a state in the merged automaton is a set of NFA states drawn from all patterns at once, so one pattern’s accept condition can be reached through state shared with another, producing matches the per-pattern incumbent never makes. On our correctness suite the merged design passed only 4 of 13 cases (31%), and it cannot be patched without per-pattern submatch tracking—which removes the very sharing that made it fast. We enforce the gate with an oracle that compares the candidate’s output to the pure-Ruby gsub reference byte-for-byte over our input corpus (§4); a single mismatch fails the candidate.

Gate 3: thread-safety in a managed runtime. The engine runs inside a garbage-collected host with multiple threads and a global interpreter lock that we want to release on large inputs (to let other threads run during a long scan). Several fast prototypes kept mutable matcher state in process globals—including, subtly, a lazy DFA cache that is filled *during* a scan—which is a data race the moment two threads scan at once or the lock is released mid-scan. The gate is what forces the immutable/mutable state split described in §5.4; until that refactor, the otherwise-shippable engine was ineligible.

The objective: robustness across all match densities. Among the candidates that pass all three gates, we do not optimize average-case throughput on a representative payload. We optimize the worst case across match density, because a redaction library cannot choose its input: the same code path must handle a log line with one secret in ten kilobytes and a credential dump that is almost entirely secrets. An engine that is fastest on typical text but degrades on dense input fails the use case even though it would win a benchmark average. This objective is why the density sweep (§5.2) is the decisive measurement rather than a single headline number, and why a design whose cost is flat across densities is preferred over a faster-on-average one that is not.

4 METHOD: THE PROTOTYPE SEARCH

TODO.

5 FINDINGS

5.1 The incumbent is slow for engine reasons, not for being C

The starting point is a result that looks, at first, like an indictment of the implementation language: the C extension is 3–5× *slower* than an equivalent pure-Ruby loop that calls `String#gsub` with the same pattern set. A C extension is normally expected to beat the interpreter it extends, so this inversion is the paper’s entry point. We argue it is a category error to read it as “C is slower than Ruby here.” The controlled variable is the regex engine, not the language: the C path drives glibc’s POSIX `regex`, while the Ruby path drives Onigmo, Ruby’s own engine. The two engines differ in two ways that both favour Onigmo on this workload.

First, a pre-filter gap. Onigmo applies a Boyer–Moore literal scan before committing to NFA evaluation: at a position with no required anchor byte it skips ahead by the shift table. Many redaction patterns carry long literal prefixes (`sk_live_`, `AKIA`, `--BEGIN`), so on ordinary text Onigmo rejects most positions cheaply, whereas glibc `regex` evaluates the automaton at every position. Second, an allocation gap. glibc `regex` allocates state proportional to the input length on each call, while Onigmo evaluates against a fixed-size stack; with one pass per pattern, that per-call cost is paid once per pattern per input. Neither factor is about C.

These two effects are not independent of density, which sets up the rest of the paper: the pre-filter advantage is largest exactly when hits are rare (most positions can be skipped) and shrinks as hits become common. Figure 1 places the reproduced pre-v19 glibc engine (§4) against the field across the density range; in our configuration it sits roughly an order of magnitude above every shippable engine, including the pure-Ruby loop it was meant to replace.

5.2 The pre-filter pipeline penalty grows with match density

The conventional move for multi-pattern matching is to put a shared literal pre-filter in front of a confirmation engine—an Aho–Corasick or Boyer–Moore pass that finds candidate positions, followed by a full regex check only there. We built that pipeline (Aho–Corasick + Boyer–Moore + PCRE2 JIT) and measured it against the same PCRE2 JIT engine with no pre-filter, across a continuous sweep of match densities (Fig. 2, Table 1).

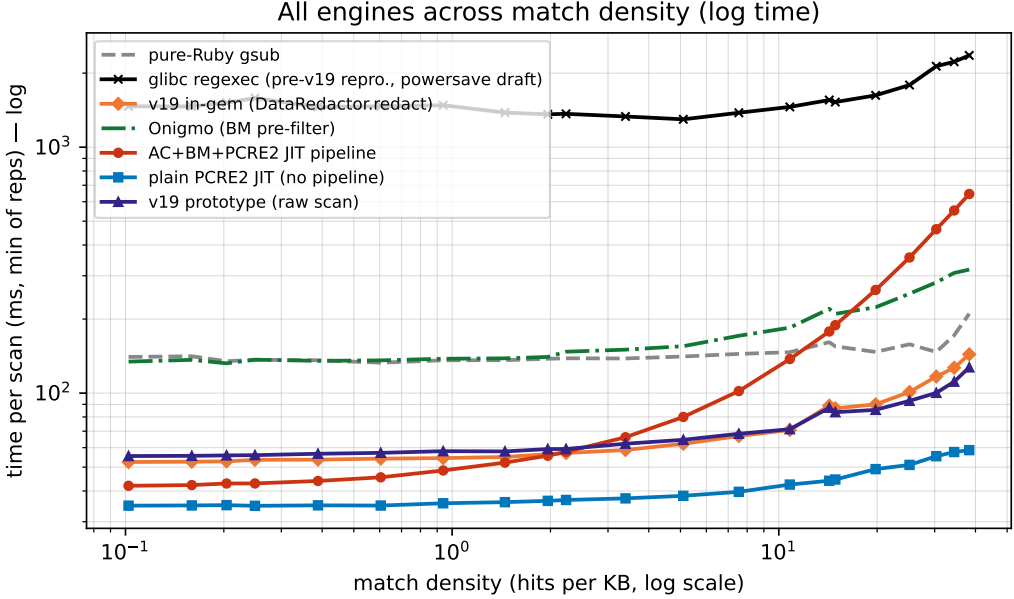


Fig. 1. Time per scan vs. match density (log–log), all engines. The reproduced pre-v19 glibc regexec engine sits roughly an order of magnitude above every shippable engine—slower than the pure-Ruby gsub loop it was meant to beat. *Two operating points*: the shippable engines are from a clean run (idle machine), while the glibc baseline is reused from an earlier powersave run, as re-measuring the slow incumbent across the dense strides is impractical (§7); this only understates the gap, since the fast engines ran with more CPU headroom than the baseline.

The mechanism is general: a literal pre-filter pays off only by *rejecting* positions before the expensive engine runs. Its benefit is therefore bounded by how many positions it can reject, and that fraction falls as matches become denser. Past some density the pre-filter inspects nearly every position, finds a candidate at most of them, and so adds its own cost on top of a confirmation step it can no longer avoid. The pre-filter becomes pure overhead.

The magnitude, in our configuration, is sharper than the mechanism alone predicts. The pipeline is not merely worse at high density—it is slower than the same JIT without the pipeline at *every* density we sampled, from the sparsest (≈ 0.1 hits/KB) to the densest (≈ 38 hits/KB). The penalty grows monotonically with density, from $1.2\times$ slower to $11\times$ slower. Onigmo, which also fronts its automaton with a Boyer–Moore scan, rises with density on the same payloads—independent corroboration that the effect is a property of pre-filtering, not of one implementation.

The single sentence we take to §6 is the crossover against the incumbent the pipeline was meant to beat: *the optimized C pipeline overtakes the pure-Ruby gsub loop at ≈ 12 hits/KB*—below that density it wins, above it the hand-built pipeline is slower than the Ruby it was built to replace. Plain PCRE2 JIT and the v19 lazy-DFA engine (§5.3) show no such climb; they stay flat across the whole range, which is what a design selected for worst-case density robustness must do.

5.3 The shippable design: per-pattern lazy DFA + two selective merges

TODD.

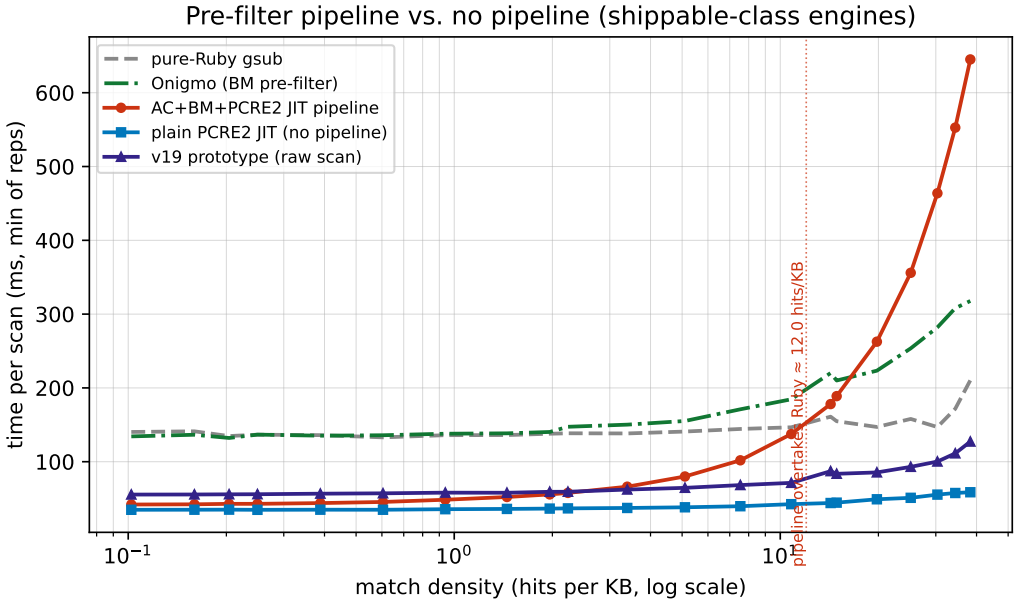


Fig. 2. Time per scan vs. match density (shippable-class engines, linear y ; clean run). The AC+BM+PCRE2-JIT pipeline tracks the no-pipeline engines when hits are rare, then its cost rises with density and overtakes the pure-Ruby baseline at ≈ 12 hits/KB. Plain PCRE2 JIT and the v19 lazy-DFA engine stay flat. Relative to plain PCRE2 JIT, the pipeline ranges from 1.2 $\times$ slower (sparsest) to 11 $\times$ slower (densest) in our configuration.

Table 1. Time per scan (ms, min of reps) at selected densities. Shippable engines from the clean run; the glibc baseline column is the power save draft (§7). Generated by `paper/plot_density_sweep.py`.

hits/KB	pure-Ruby gsub	glibc regexec	v19 in-gem	Onigmo	AC+BM+PCRE2 JIT pipeline	plain PCRE2 JIT	v19 prototype
38.28	210.3	2360.5	143.7	317.8	645.1	58.6	127.2
14.27	161.0	1554.9	88.9	220.2	178.1	44.0	87.2
1.96	137.9	1360.3	56.4	140.3	55.7	36.5	59.2
0.20	134.8	1516.2	52.8	132.2	42.9	35.0	55.8
0.10	140.3	1466.2	52.5	134.3	42.0	34.9	55.5

5.4 Prototype-to-production hardening: GVL-free re-entrancy

TODO.

6 EVALUATION

TODO.

7 DISCUSSION AND THREATS TO VALIDITY

TODO.

8 RELATED WORK

TODO.

9 CONCLUSION

TODO.

ACKNOWLEDGMENTS

The author used Claude (Anthropic) for drafting, literature triage, and code/benchmark implementation; the author reviewed and is responsible for all content.

REFERENCES